# Project Statement and Solution Approaches

## Sponsors:

- Bonnie Schnitta, SoundSense
    - Dr. Bonnie Schnitta is the founder and CEO of SoundSense, LLC, a design-build acoustical engineering firm. She holds degrees in mathematics, mechanical engineering (M.S., Tufts), and signal processing, more than 20 patents, and is a Fellow of the Acoustical Society of America and a Life Member of IEEE
    - Her role on this project is mentor and technical reviewer: checking progress, pressure-testing methodology, and giving guidance on project management aspects of the project
- Sam Sheng, Google
    - Dr. Samuel Sheng is Director of Sensing Technologies at Google and an expert in systems design and digital signal processing, with extensive experience architecting CMOS ASICs for ultrasonic sensors, silicon tuners, DSL transceivers, and low-power mobile devices. He co-authored the landmark paper "Low-power CMOS digital design" (IEEE JSSC, 1992). As a mentor, he will review the accelerator architecture and our energy measurement methodology, and advise on the robot's sensing.

**Goal:** build the accelerator in the Kria fabric, the datapath between it and the CPU, and the wrapper the chess engine calls into. Ideally, this runs faster than the built in DPU that the SoC currently has.

**Problem Statement:**

**How good can a robot’s decisions be on cheap hardware? We use chess to measure it.**

More and more robots need to make their own decisions using cheap onboard computers, with no internet connection. Cheaper hardware means weaker AI, but it’s hard to know how much weaker, because most robot tasks have no clear way to score a decision as good or bad.

Chess does: every move can be scored, either against strong engines or against what a human at a given rating would play. We’re building a chess playing robot as a test case to measure how decision quality changes when all of the AI has to fit on a single $349 board, and which design choices recover the most of it. The robotic board is the open, low-cost platform; the accelerator is the research we demonstrate on it.

Additionally, on-Device AI (Edge AI) is rapidly growing. It is an important topic to get our hands in because of its potential prevalence in the future. Local processing cuts down latency, protects private user data on the device, and reduces recurring cloud computing costs for simple tasks. Smartphones, PCs, wearables, drones, and smart-home gadgets now run lightweight local models (such as Apple Intelligence or Gemini Nano).

## Who is the user?

- **Engineers and researchers** who are deciding whether to build a custom accelerator or use vendor IP for small, latency-bound networks. They need measured latency, throughput, energy, and resource data.
- **A human player** who plays a complete game against the robot without a keyboard. This is the human-centered face of the project and the showcase at the senior design expo.

## How will we gather requirements?

- Biweekly alternating check-ins:
    - Bonnie to confirm goals, review test plans, and sign off on what "done" means for each milestone
    - Sam to review of the accelerator architecture and measurement methodology
    - Early agreement between the network and hardware leads on the network specification (layer types, sizes, quantization), since it sets the accelerator design
- Literature review of FPGA inference, AMD's DPU, and neural chess engines to set baselines
- Low-cost robot prototypes to test the riskiest mechanism assumptions before committing
- Playtesting with Tufts students and chess club members to find usability problems
- A written requirements list with each item marked mandatory or stretch, signed off by the sponsor at the end of fall

## What is the functionality?

- Detect the human's move with RFID after a turn signal (button or clock press)
- Check the move is legal and signal the player if it is not
- Choose a reply with the Maia network running on the FPGA accelerator (one inference per move, no tree search).
- Physically execute the reply, including captures, castling, en passant, and promotion
- Verify each robot move with RFID sensing; retry once, then ask the human for help

# Chess Engine Accelerator

### What we're building

| Piece | What it is |
|---|---|
| **Custom integer accelerator in RTL** | The math hardware: a matrix-multiply engine plus a special-function unit for softmax, layer norm and attention bias |
| **AXI control registers** | Memory addresses the CPU writes to, to start the accelerator and read back status |
| **DMA integration** | Xilinx IP that moves data between DDR and the accelerator without the CPU copying bytes |
| **AXI-Stream interface** | The handshake at our accelerator's edge. The DMA talks to memory for us, so **we don't write bus-mastering logic** |
| **Processor-side wrapper** | Software on the A53s that drives all of the above and gives the chess engine one function to call |
| **Hardware verification** | Proving the RTL computes the right numbers before trusting it on a board - still figuring out exactly what we need here |
| **Performance characterization** | Measuring on real hardware, especially against the DPU IP |

## Platform - AMD Kria KR260

**Kria KR260 Robotics Starter Kit, $349.** Zynq UltraScale+ MPSoC on the K26 SOM.

- Quad Cortex-A53 at up to 1.5 GHz
- Dual Cortex-R5F real-time cores
- 4 GB DDR4
- Programmable logic: 1,248 DSP slices, 23.1 Mb on-chip memory

### Why we think it's a good fit

1. **Enough on-chip memory to hold a complete quantized network.** This is the premise of the whole design - arithmetic below.
2. **Enough DSP slices for a MAC array.** 1,248 of them, each doing 2 INT8 multiply-accumulates per cycle.
3. **Xilinx/Vitis compatibility and IP we can pull from.** The DPU gives us a working baseline to compare against, and the DMA and interconnect blocks are free.

4. **Host processors run the game logic, board sensing and accelerator wrapper, leaving the fabric free for inference.**
5. **Robotics carrier I/O** - Pmods, RS485, CAN, extra Ethernet.

### Other options we considered

| Option | Why not |
|---|---|
| **KD240** | No UltraRAM, and 3.5× fewer DSP slices |
| **PolarFire SoC** (Matthew's suggestion) | Way out of budget at ~$2k |
| **Microcontroller** | Gets rid of the accelerator portion entirely |

## On-Chip Memory Budget

The K26 has two kinds of fast memory built into the fabric.

| | BRAM | UltraRAM |
|---|---|---|
| Blocks | 144 | 64 |
| Per block | 36 Kb (kilo**bits**) | 288 Kb |
| **Total** | **648 KB** | **2,304 KB** |

> ***Combined: 2,952 KB = 2.88 MB***

Note the specs are in kilo**bits**, so there's a /8 to get bytes. Working example: 64 UltraRAM blocks x 288 Kb x 1,024/8 = 2,359,296 bytes = 2,304 KB.

**UltraRAM** is the bulk store - big, dense, fixed-width. Weights go here.

**BRAM** is the flexible one - configurable width, built-in FIFO logic. Activation buffers, AXI FIFOs, and small tensors go here.

### Getting to a usable number

That 2.88 MB is 100% utilization. Aim for ~90% (a guess for now) to consider other potential requirements uncounted:

> ***≈ 2.6 MB placeable***

Then we need room for things that aren't weights: activation buffers between layers, AXI FIFOs, control logic, accumulators.

> ***Limit weights to 2 MB INT8 ≈ 2 million parameters***

The gap between 2.6 and 2.0 is our working room. At INT8, one parameter is one byte, so the parameter count and the byte count are the same number.

## What We Need From Matthew

**The neural network architecture specification** - layers, channels, kernel sizes. Our layer sequencer is a state machine that walks the network in order, so it can't be built against an unknown shape.

**A model that fits in 2 MB of INT8 weights. (Do we think this is achievable ??)**

We also need the artifacts that make the hardware produce *correct* numbers - weight byte layout, quantization parameters, golden test vectors to check our outputs against. Those we can work out later; the architecture spec is what unblocks us first.

Total parameter count of the transformer student, so we can check it against the 2 MB budget.

His reference implementation must use the same integer approximations for exp, reciprocal and square root that our hardware uses, or our outputs can never match his exactly.

## Why a Custom Accelerator Instead of the DPU IP

*(Other than it's cool.)*

Our model is a transformer. The onboard DPU (Deep Learning Processing Unit) is built for convolutional networks like ResNet and, as far as we know, does not run attention or softmax. Those layers would fall back to the ARM CPU, sending data out through DDR and back. We will confirm this with the Vitis AI compiler's partition report, which lists every operation the DPU can't run.

The model uses chess-specific operations such as geometric attention bias (GAB): a learned value added to each attention score based on how two squares relate on the board (same file, same diagonal, a knight's move apart). No general-purpose accelerator supports this. In our fabric it is a small lookup table and an adder.

The DPU is locked to INT8. Softmax and layer norm usually need more precision than that to stay accurate, while most weights could use less. Our hardware can use the right precision for each part of the model.

The DPU streams weights from DDR. If our model fits in on-chip memory, we load the weights once at startup and never fetch them again, saving time and energy on every inference.

Because Maia makes one inference per move, our advantage shows up mainly in latency, energy, and which operations stay on the fabric, rather than in playing strength.

## Architecture

All weights load into on-chip memory at startup. A single MAC array is time-multiplexed across layers, with activations ping-ponging between two buffers.

The design has two blocks: a matrix-multiply engine for the projection and MLP layers, and a special-function unit for softmax, layer norm and the geometric attention bias.

**Time-multiplexed** means we build **one** MAC array. Compute layer 1, reload it with layer 2's weights, compute layer 2. We reuse it as many times as we need.

Idea is we have layer 1's output become layer 2's input, read from buffer A, write to buffer B, then swap and alternate. Jumps back and forth for however many layers we have.

### 64 lanes

A single weight value can go to all 64 lanes simultaneously. The model treats each of the 64 squares as a token, and the projection and MLP layers apply the same weight matrix to every square, so one memory read feeds 64 multiplies.

### Future option: pipeline per layer

Instead of one MAC array reused for every layer, build a separate array per layer and pipeline them - each working on a different position at the same time, like an assembly line.

More throughput, but it needs a buffer between every stage instead of two, which eats the weight budget. **We can look into this later.**

## Testing

We'd compare against the DPU. Same network, same quantization, same positions - one running on the DPU IP, one on our accelerator. We run the same INT8 model on both first, to isolate the effect of where the weights live, then show mixed precision as a second result.

## Open Questions

**What if the model exceeds budget?** Worst case, stream layer N+1's weights from DDR while layer N is being computed. That keeps weight reuse *within* a layer even if the whole network doesn't fit. Question is more about whether there's a better fallback.

**Do you have any advice for how we should structure our architecture?**

**Any other recommendation for SOC we should use?**

**Robotics**

**Problem Statement:**

In the ever changing world of AI, robotics have been in the forefront in the application of it. From warehouse robots to robots made by hobbyists, robots are integral to research and development and how we can apply AI to the real world. Unfortunately, the hardware of robotics can inhibit this development. Robots are expensive or sometimes not interfaceable to be connected with every single AI model.

Chess is an ideal small-scale version of this problem. The state is fully defined, every action has a clear correct outcome, and success is unambiguous. Doing it reliably still requires the hard parts of general robotics: identifying objects, precise pick-and-place, planning actions around obstacles, recovering from failures, and coordinating perception, decision-making, and motion in real time with a person in the loop. We will build an open-source, low-cost pick-and-place robot that uses chess as a structured benchmark.

**Overview**

We will build a Cartesian XYZ gantry that moves pieces with a gripper, on a custom board that identifies every piece using resistor-coded bases.

The robotics subsystem has three jobs:

- **Move pieces.** Execute the engine's move on the physical board, including captures and special moves.
- **Sense the board.** Each square reads the resistor in the piece above it, so the board knows which piece is on every square. This detects the opponent's move and verifies the robot's own moves.
- **Talk to the rest of the system.** A state machine with a handshake connects the engine, board sensing, and motion control.

We will keep mechanical complexity low and spend most of our effort on software, control, and integration with the engine.

**General Resources**

The Nolop and Bray makerspaces at Tufts provide basic supplies, 3D printers, laser cutters, and metalworking machinery. We will 3D print the pieces to keep costs low and to embed the resistor, contacts, and weights directly in each base. We want to keep the robotics budget below $100, as the main expense is the SoC.

**Draft Requirements**

These targets are first guesses so we have something to measure against. We expect to revise them after prototyping.

| Requirement | Draft target | Notes |
|---|---|---|
| Move time | Under 10 s per move | Captures and castling can take longer |
| Placement accuracy | Within ±5 mm of square center | Squares are roughly 50 to 57 mm |
| Reliability | 50 consecutive moves with no intervention | Our main demo metric |
| Sensing accuracy | Correct piece identity on every square, every scan | Ambiguous or unreadable squares are flagged, never guessed |
| Workspace | Full 8×8 board plus a capture zone | Board is roughly 450 to 500 mm across |

| | | |
|---|---|---|
| Z clearance | About 150 mm of travel | King is about 95 mm tall on a standard set |
| Cost | Less than $100 | Built with 3D printers and makerspace machinery |
| Footprint | Fits on a table, portable | |

**Gantry design**

A gantry frame straddles the board and moves a gripper in X, Y, and Z. It is essentially a 3D printer with a gripper instead of a nozzle, so most parts are off the shelf.

How it works:

- X and Y axes on belts (GT2 belts, NEMA 17 steppers); Z on a lead screw for precise vertical positioning.
- The gripper descends, grabs the piece, lifts clear of all other pieces, travels in a straight line, and places it.
- Because pieces are lifted over the board, there are no collisions, and knight moves are no harder than any other move.

Workspace (draft, in mm, origin at board center):

- X: about −300 to 300 (board plus a capture zone on one side)
- Y: about −260 to 260
- Z: 0 to about 150

End effector options:

- **Two-finger servo gripper:** needs a jaw design that handles pawn to king. 3D-printed pieces can share a standard neck profile to make this easier.
- **Electromagnet:** simplest and most reliable; a steel slug in each printed piece gives it something to grip and adds weight for better contact.
- **Suction cup:** needs flat-topped pieces, which we can design into the printed set.

Pros:

- Simple kinematics: each axis is independent, with no inverse kinematics.
- Mature parts and firmware patterns from 3D printing.
- Pieces are placed straight down, which is ideal for contact-based sensing.

Cons:

- The gripper is the hardest mechanical problem.
- The frame sits over the board, which may bother a human player.
- Three axes plus a gripper means more motors and more calibration.

**Board sensing: resistor-coded pieces**

Each piece has a resistor in its base connecting two contacts: a center dot and an outer ring, so the piece reads correctly at any rotation. Each piece type and color gets its own resistor value. Each square has matching pads wired as a voltage divider with a reference resistor. An empty square reads near 0 V, and each piece reads its own voltage level.

Four 16-channel analog multiplexers route all 64 squares to one analog-to-digital converter. A full scan takes about a millisecond. Software keeps the game state and checks that sensed pieces match legal moves, so a bad reading is flagged rather than played.

**Shared components**

*Turn detection:* the player presses a button or chess clock when their move is complete. Turn detection will be player input.

*Special moves:*

- **Captures:** remove the captured piece to the capture zone first, then move the capturing piece.
- **Castling:** two moves, king then rook.
- **En passant:** the captured pawn is not on the destination square.
- **Promotion:** spare queens are staged in the capture zone, and the board reads which piece the player placed.

Because we know where all the pieces will be at any given time, we can determine if a player makes an illegal move.

*Interface and state machine:* motion commands reuse UCI notation plus flags. Every message gets an ACK, so a dropped message is detected rather than silently hanging the system.

*Calibration and homing:* limit switches on each axis home the gantry at startup. A one-time calibration jogs the gripper to the corner squares to map board coordinates.

*Failure handling and safety:*

- A missed pickup, dropped piece, or knocked-over piece shows up in the post-move board scan. The robot retries once, then asks the human for help.
- The gantry only moves after the player presses the turn button, and an emergency stop button cuts motor power.

**Plan and open questions**

Tentative plan:

1. Build a cheap proof of concept for the two riskiest parts: a gripper that reliably picks every piece type, and resistor-coded pieces that read correctly on every square.
2. Build the full gantry and reach basic functionality (move one piece square to square).
3. Add captures, special moves, and post-move verification.
4. Integrate with the engine through the state machine, focusing on reliable, low-latency messaging.
5. Iterate against the reliability target.

Questions:

1. We have considered using simulation software such as Gazebo before prototyping, but simulating may be a waste of time, and we could instead begin prototyping and testing directly. Which path best fits our needs?
2. How accurate does the robot need to be for any given piece? What would hold us back on accuracy the most: mechanical hardware or software?
3. Is there any value in using ROS? It works well with Gazebo, but beyond that, what are its costs and benefits?
4. For resistor-coded pieces, what contact design and resistor spacing would you recommend so readings stay reliable when people place pieces slightly off-center?

**Ethical Dilemmas:**

Despite its simple and innocuous goal, the project raises a few real ethical tensions. Making the board open-source and easy to connect to any model is its main value to the general public. That same openness could help someone build a concealed engine-assist device for cheating in over-the-board play, so we have to weigh accessibility against misuse. Our central claim, that a custom accelerator beats AMD's DPU, creates pressure to tune the comparison in our favor. Honest science requires identical test conditions and reporting results even if they don't support our hypothesis. Finally, a robot that moves parts where people's hands go, shown at a public expo where children may handle small magnetic pieces, forces a tradeoff between an open, approachable demo and the guards, interlocks, and enclosed parts that keep players safe.

**Major Changes:**

**Chess Engine Accelerator**

**We have come up with some potential solutions to shrink the model more:**

- We could make the student smaller by further reducing embedding width, MLP width, or layer count and distill from Maia-3. While this can further save weight storage beyond the already shrunken model, the quality has to be preserved (solve for quality not just fitting)
- Recurrent student where we reuse one or two transformer blocks over multiple iterations and could potentially several-fold fewer weights, though repeated computation still costs time and equal quality is unproven. This is especially hardware-friendly: reusing one block's weights across iterations is exactly what on-chip memory rewards.
- Factorized or selectively shared architecture where we replace large matrices with low-rank factors or share only selected attention/MLP weights. This potentially offers a lot of intermediate compression with more flexibility than full recurrence.

**We have come up with some potential solutions just in case we aren't able to fit the model onto the on chip memory:**

- We can beat it if their dpu is optimized for a certain fp8 or fp16 or int8. If it's not, we can beat it by making it super specific for this. Sam's idea
    - The Zynq UltraScale+ DPU is INT8 only, for both weights and activations. No FP8, no FP16, no INT4. Every model gets squeezed into INT8 whether that's the right fit or not.
    - The DPU is locked to INT8, but our hardware can use whatever precision the model actually needs, such as INT4 or mixed precision per layer. Narrower integers cut memory and fit more multiplies per DSP, raising our peak compute above the DPU's, provided Matthew's model keeps its accuracy at lower precision. We'd stick to integers rather than FP8 or FP16, since FPGAs have no native floating-point units and float multipliers cost far more logic. Softmax and layer norm typically need more than INT8 to stay accurate, so mixed precision is likely required anyway: INT4 or INT8 for most weights, wider precision for those operations.
- Double buffering: Split on-chip memory into two weight slots. While the array computes layer N from slot A, the DMA loads layer N+1 into slot B. Then swap.
    - If computing a layer takes longer than loading the next one, the load is fully hidden and the array never waits. This hides most of the cost of streaming weights from DDR.
- DDR-backed tiled accelerator where we stream weight tiles using packed bursts, double buffering, and prefetch
    - Removes the all-on-chip size constraint, but introduces bandwidth and energy costs.
- Hybrid residency and execution where we keep frequently reused weights/state on chip, stream the remainder, and optionally execute selected operations on the ARM CPU. This could greatly reduce FPGA memory demands, but the transfer and CPU overhead is significantly larger and must be measured end to end.

No real other changes other than our first plan of action is measuring DPU baseline first to compare.

Reminder: what are we actually comparing?

- Can we quantify the degradation of the model per size shrinkage: **Quantify how much model quality degrades as the model shrinks, connecting the hardware results back to our problem statement.**
- Accuracy to Original Model Performance via standardized test datasets
    - We can also measure how accurate the model predicts the exact move a human player makes of the targeted Elo
- Latency per move at batch 1: Maia makes one inference per move, so this is the speed that matters, and it's where the DPU's per-call overhead hurts most
- Energy per inference (Sam is reviewing our energy measurement methodology)
- Which operations run in fabric versus falling back to the CPU
- Resources used (DSP slices, BRAM, UltraRAM), so the comparison is fair

**Chess Robot:**

Based on feedback from our sponsor, mentor, teaching team, and teammates, we made the following updates:

1. Dropped computer vision for a resistor-coded board. Each 3D-printed piece has a resistor in its base, one value per piece type and color, which an ADC reads on every square. The CV was too complex and outside our scope. An occupancy-only board would have to infer moves, which is fragile around captures, promotion, and error recovery. Resistor coding, suggested by our mentor, gives reliable piece identity at very low cost.
2. Chose the overhead gantry and removed the under-board CoreXY design.*:* the board's underside now holds the sensing pads and wiring, so a mechanism under the board is no longer practical. We are still choosing between an electromagnet and a gripper, and will decide with a quick pickup test.
3. Custom 3D-printed pieces and board. Each piece holds its resistor, contacts, and a weight, and every piece shares a common top for pickup. This keeps costs low, improves contact reliability, and simplifies pickup. It also resolves our open question of whether the robot should work on any board.
4. Verification, safety, and calibration redesigned without a camera. The board is rescanned after each robot move. The gantry moves only after the turn button is pressed, an emergency stop cuts motor power, and limit-switch homing plus corner-square calibration replace camera fiducials.
5. Problem reframed. The project is now an open-source, low-cost robotic chessboard that any engine or AI model can connect to, with our FPGA-accelerated engine as the demonstration. Commercial robotic boards are expensive and closed, and this framing separates the board's value from the accelerator research. Our value is bringing a flexible and interfaceable board at a price point that is available to many more people.